

# NYC Green Taxi Revenue Pipeline

Daily revenue for 146,475 trips, calculated three ways

A data pipeline project from my AI engineering training at neuefische

**Michel Mghames**

# The task

1

## Download

The green taxi files for January to March 2025, into a staging folder

2

## Calculate

The revenue per day, with an ETL or ELT pipeline

3

## Bonus

Run the pipeline with Prefect



**146,486**

trips in three monthly files, 21 columns each

The calculation needs only two columns: the pickup time and total_amount

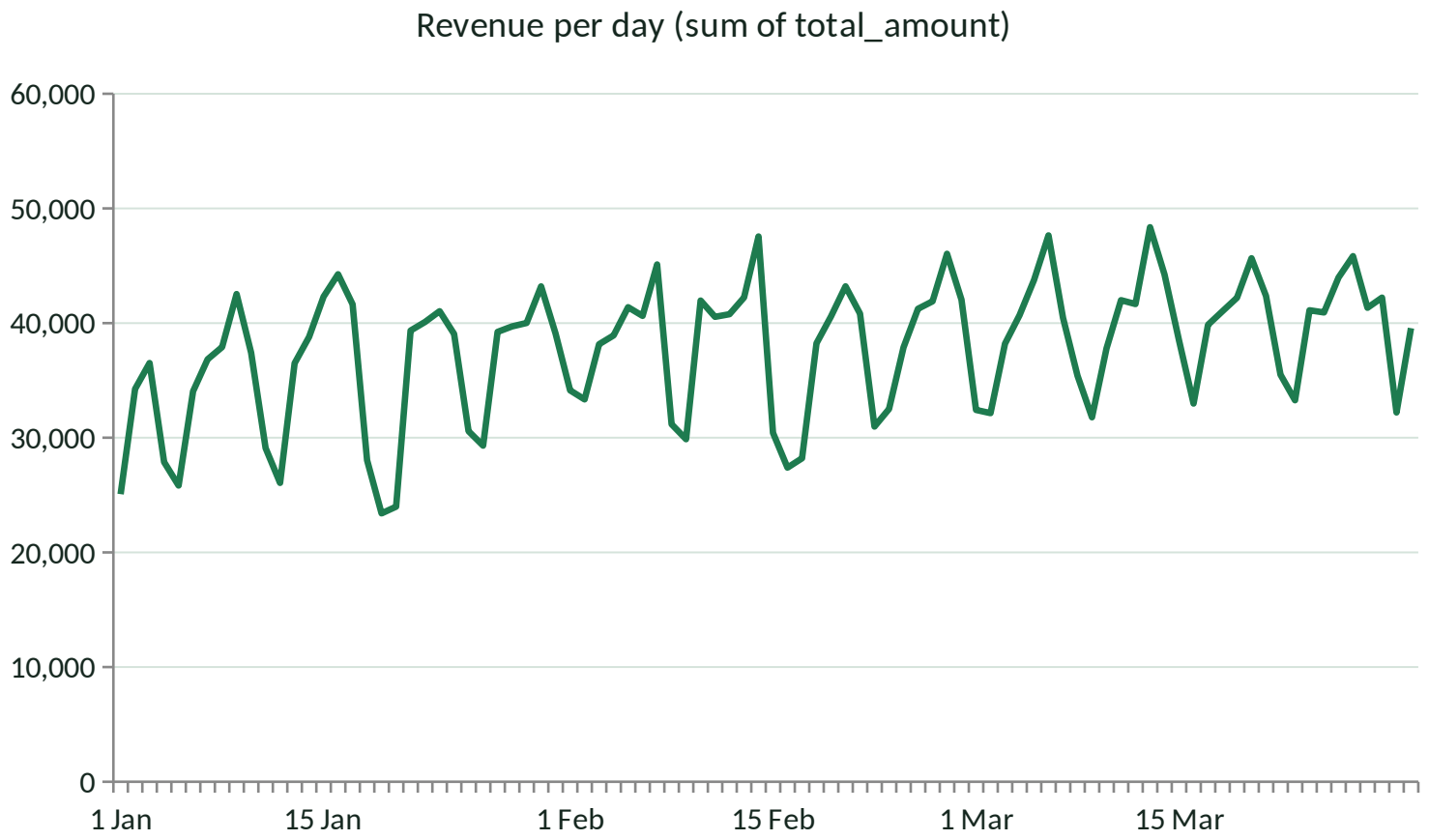
What I added: three engines, an exact comparison, 89 tests and a written study

# The result

**146,475**
trips used

**90**
days, 1 January to 31 March

**3,395,290.56**
total revenue, the sum of total_amount




Weekdays earn more than weekends. The best day was 13 March (48,362.43).

# How the pipeline works

## Extract

**download.py**

Files are saved unchanged in data/raw, the staging folder

## Transform

**three engines**

pandas, polars and DuckDB apply the same three rules

## Load

**pipeline.py**

daily_revenue.csv, daily_revenue.parquet and pipeline_metadata.json

```
uv run taxi-revenue   download   |   run   |   compare   |   flow
```

Four commands. The engines return the same two things: a table with one row per day, and a dictionary of counts.

# The real data came first

**11**

**trips outside January to March**

dropped, worth 304.73 in total

**478**

**negative amounts**

-6,462.61 together. 464 of them are no-charge or dispute trips

**254**

**zero amounts**

mostly from one data provider, vendor 6 (see slide 12)

I looked at all three files before writing any calculation. The rules come from what I found.

# Three rules, each with a reason

1

## Date window

Only trips from 1 January up to, but not including, 1 April 2025 count.

**11 trips dropped**

2

## Keep negative and zero amounts

The result is net revenue. Most negative amounts look like corrections.

**478 and 254 kept**

3

## Drop unusable trips, and count them

A trip with no date or no amount is unknown, not zero.

**0 in this data**

Each rule is counted in the metadata file, so nothing disappears quietly.

# Money in whole cents

### As decimals

```
0.1 + 0.2
```

```
= 0.30000000000000004
```

The last digits can differ, depending on the order of the additions

### As whole cents

```
10 + 20
```

```
= 30
```

Integers add up exactly, in any order

So pandas, polars and DuckDB can reach exactly the same number. In this data, no amount has more than two decimals, so nothing is lost.

# Three engines, two styles

**pandas | ETL**

- Reads each file into memory
- Transforms in Python, then saves only the result
- Default engine: simplest to read

**polars | ETL, lazy**

- Builds a plan first, runs it at the end
- Reads only the 2 columns it needs
- Fastest engine in my tests

**DuckDB | ELT**

- Loads the raw files into a table first
- Then 5 SQL steps run inside the database
- Every step is a table you can look at

ETL: transform before saving.  ELT: load first, transform inside the database.

# Proof: the results are identical

1. Result tables compared exactly, with no rounding tolerance
2. CSV files identical, byte for byte
3. Counts (metadata) identical
4. 150,000 random trips compared in a test

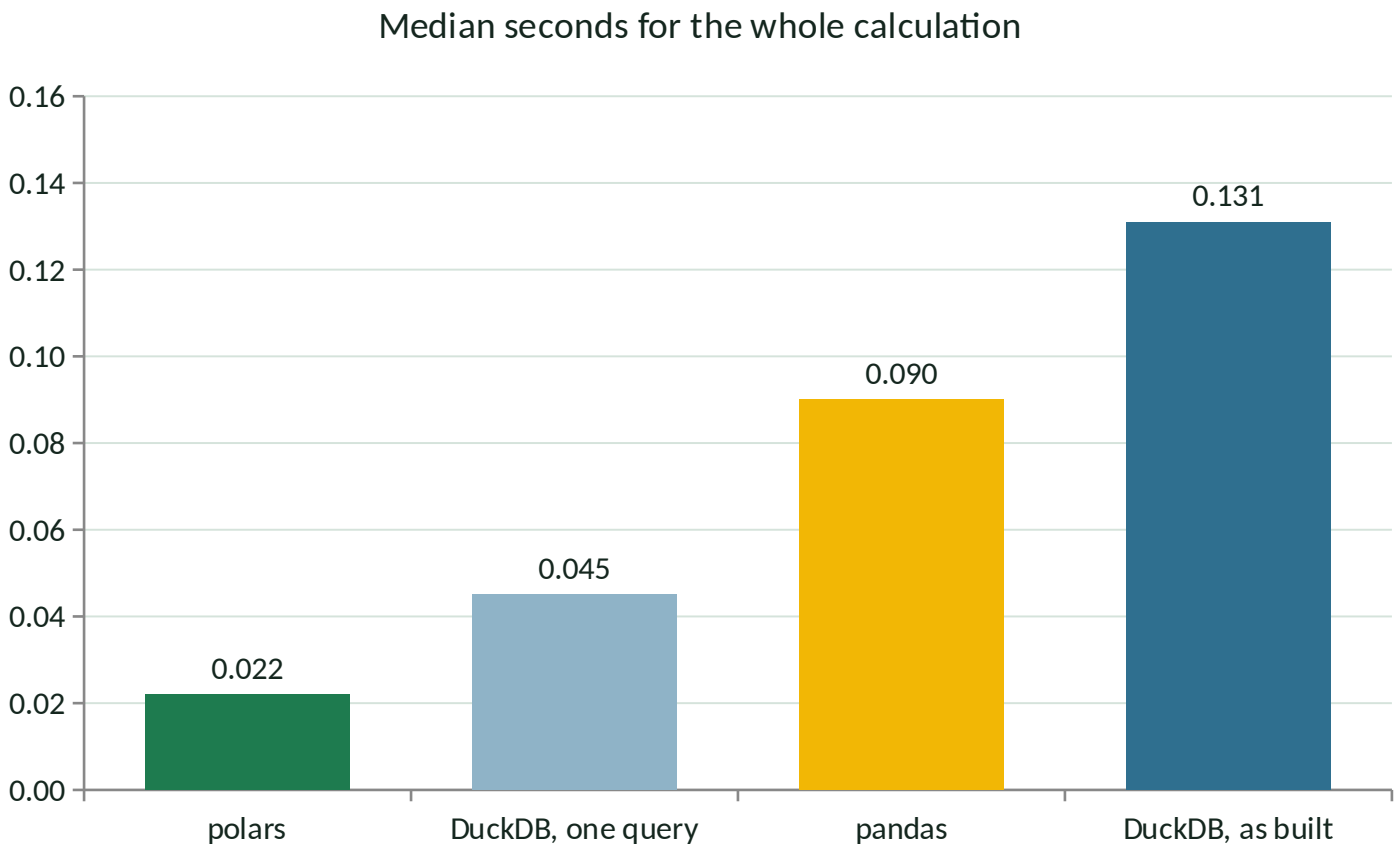


**`573c923d786f`**

The CSV fingerprint (first 12 characters of the SHA-256 hash) of pandas, polars and DuckDB

My first check was too tolerant and missed a planted error. The exact check fixed that.

# How fast is each engine?




## 4.1 times

polars is faster than pandas as built

## 2.9 times

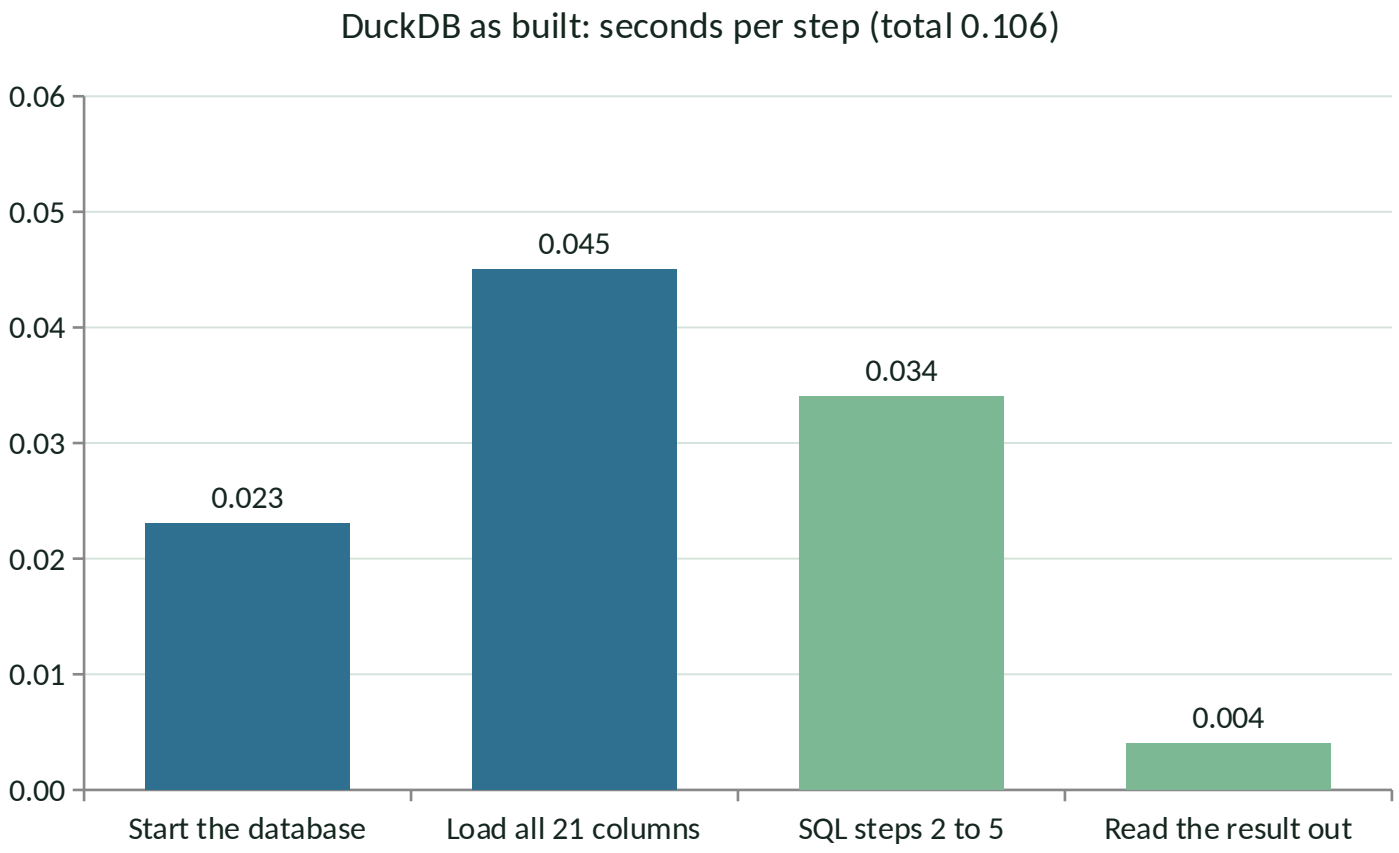
faster: DuckDB as one query, against five layers

## 35 percent

variation between two runs of the same code: below about 20 percent is noise

146,486 trips, one Windows computer, median of 7 runs after 1 warm-up run.

# Why DuckDB was slower




## 64 percent

of its time goes into starting and loading, before any SQL runs

## 0.034 s

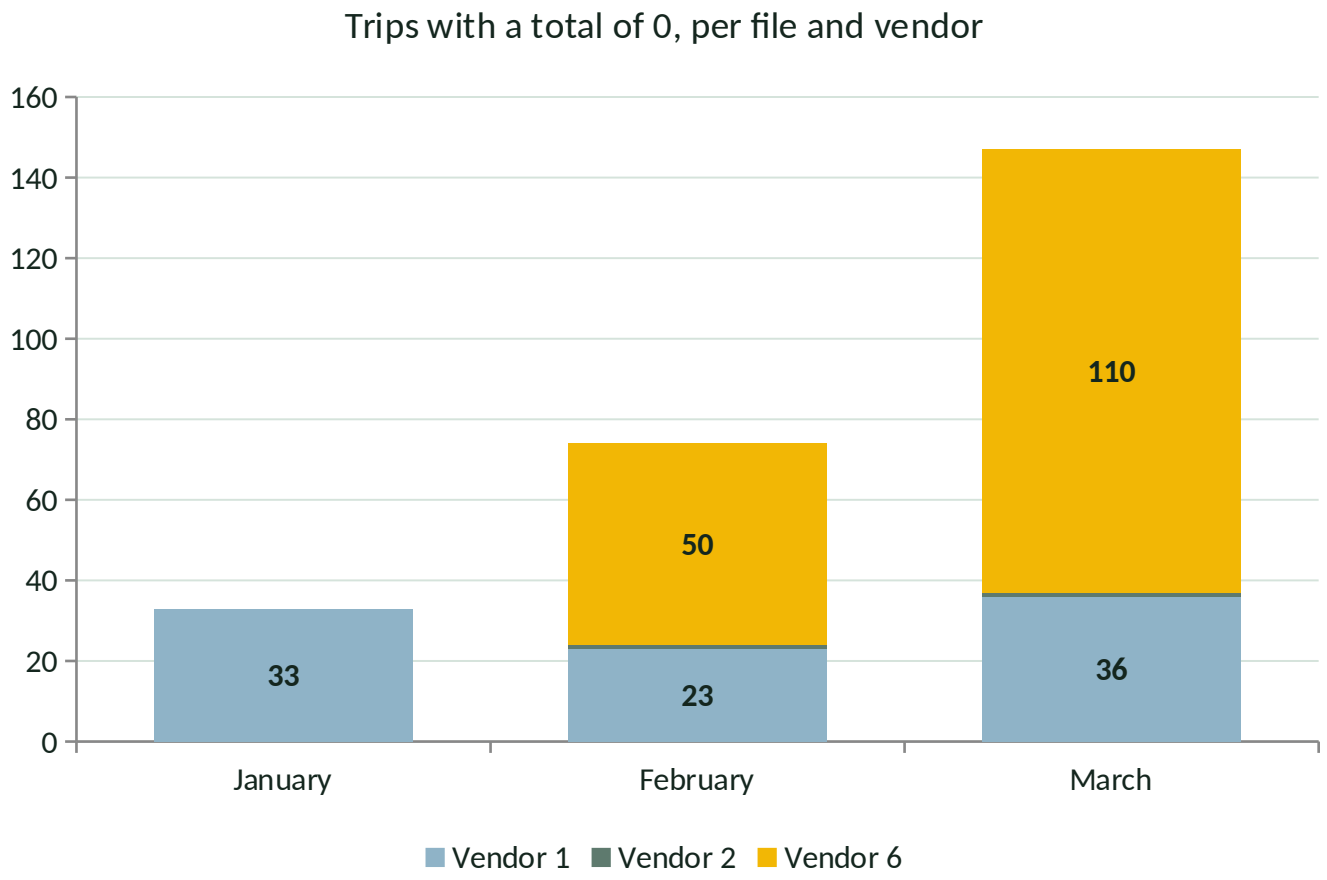
for all four SQL steps together: the layers themselves are cheap

## 0.025 s

of the 0.131 s are not explained. I did not measure that part

My first guess was partly wrong: loading mattered, the five layers did not.

# The zero amounts and vendor 6




## Vendor 6 first appears in February

It is one of three companies that send the trip records

## 160 trips with a total of 0

but with fare parts worth 677.15, which is 0.02 percent of the revenue

## Decision: no new rule

Documented openly. The reason is not known: ask the data provider

Without vendor 6, the zero totals would be 33, 24 and 37: a steady level.

# 89 tests, and how I checked them

**89**

automated tests, all passing

about 35 seconds to run

**The same checks run for all three engines**
rules, edge cases, errors and 150,000 random trips

**I broke the code on purpose, more than 20 times**
in a test copy, to see that a test fails

**Three times my tests missed it**
the test data had no edge cases. I added them

A test only protects against mistakes that its data can show. Good test data contains the edges.

# What I learned, what is still open

## What I learned

- Look at the real data first, then write the rules
- Test the test: a comparison with a tolerance hides errors
- Compare tools on the same task before choosing one

## What is still open

- Why vendor 6 reports a total of 0, and why some totals differ from the sum of their parts

The data cannot tell it: only the data provider can. The effect is 0.02 percent of the revenue, and it is documented, not hidden.

**github.com/michobc205/nyc-green-taxi-revenue-pipeline**

Everything is open: the code, 89 tests, the results and the long study.